Question Bank — Week 4

Dynamic arrays and amortised analysis (Lecture 04) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **4**

Part C — Trace the growth **6**

Part D — Proofs **7**

Part E — Find and fix the bug **7**

Part F — Write the code **8**

> ***Questions:*** *`week04-questions.md`. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M11	c	M21	a	M31	c
M02	c	M12	d	M22	b	M32	d
M03	b	M13	a	M23	c	M33	a
M04	d	M14	b	M24	a	M34	b
M05	a	M15	c	M25	d	M35	c
M06	b	M16	d	M26	b	M36	d
M07	c	M17	a	M27	c	M37	c
M08	d	M18	b	M28	d	M38	a
M09	a	M19	c	M29	a	M39	d
M10	b	M20	d	M30	b	M40	b

W4-M01 — a. `len` is the number of elements the user stored. The capacity (b) is internal; showing it would expose the implementation.

W4-M02 — c. Capacity 1 → 2 (2nd append) → 4 (3rd) → 8 (5th).

W4-M03 — b. Resizes happen when the size reaches 1, 2, 4, …, 512 — ten powers of two below 1,000 — so exactly 10 ($2^{10} = 1024$). 999 (d) is growing by one.

W4-M04 — d. Every append copies everything already there: $0+1+\cdots+(n-1) = n(n-1)/2$.

W4-M05 — a. Copies are 100, 200, 300, …: about $n^2/200$ in total — still quadratic, so $O(n)$ per append on average. A constant step only changes the constant. (d) confuses the step size with the cost.

W4-M06 — b. Fewer than 3n units of work for n appends: under 3 each.

W4-M07 — c. The append that finds the array full copies all n elements. The amortised bound does not change that single worst case.

W4-M08 — d. Amortised analysis bounds the total over **any** sequence — a guarantee. Average case (c) averages over an assumed distribution of inputs.

W4-M09 — a. $1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 < 2n$, since $2^k < n$.

W4-M10 — b. Appends that find the array full are those made when the size is a power of two: the 2nd, 3rd, 5th, 9th, 17th, 33rd, … (one more than a power of two).

W4-M11 — c. A gap between the grow threshold (full) and the shrink threshold (one quarter) means many cheap operations must happen between resizes. Halving at one half (a) leaves no gap.

W4-M12 — d. For example: full at capacity 8, append (grow to 16), pop (shrink to 8), append (grow), … — an $O(n)$ copy on every operation.

W4-M13 — a. Capacities 4, 8, 16, 24, 32, 40, 52, 64, 76, … on Python 3.13.

W4-M14 — b. `sys.getsizeof` returns the header plus one reference (8 bytes) per slot of **capacity**; subtract the empty list's size and divide by 8.

W4-M15 — c. −1 + size = 4: the last *used* slot, not the last slot (a).

W4-M16 — d. All n elements shift right — whether or not a resize is needed.

W4-M17 — a. Nothing moves; clear the slot and decrement the size.

W4-M18 — b. The copies at each resize double, so the last resize copies more than all the earlier ones together, and it copies fewer than n.

W4-M19 — c. Any factor r > 1 gives a geometric series: about $n \cdot \frac{r}{r-1}$ copies — 3n for r = 1.5 against 2n for r = 2 — while the unused space after a resize is about a third rather than a half.

W4-M20 — d. Growing from c to 2c with c + 1 elements leaves c − 1 slots unused: about half.

W4-M21 — a. `Stack` keeps its top at the end of a `DynamicArray`. `CircularQueue` and the open-addressing map use a plain `Array`; the tree uses nodes.

W4-M22 — b. One coin for the append's own write, two saved to pay for copying at the next doubling.

W4-M23 — c. Resizes on appends 2, 3 and 5 take the capacity 1 → 2 → 4 → 8; the sixth append fits. `len` is the size, 6. (a) confuses size with capacity; (d) grows by a factor of 3 from 4.

W4-M24 — a. The `Array` has all `capacity` slots (so (b) is wrong), but the ones beyond `size` are not part of the user's data. Reading one must be an `IndexError`, not a silent `None` — W4-B2 is exactly this bug.

W4-M25 — d. Appends 2, 3, 5 and 9 find the array full: 4 resizes, capacity 1 → 2 → 4 → 8 → 16. (c) gives the size as the capacity; (a) misses the 9th append, which finds 8 of 8 slots used.

W4-M26 — b. Allocate, copy, switch — Lecture 04's three steps. An `Array` cannot grow in place (a), and the dynamic array is one contiguous block, not a chain of blocks (d).

W4-M27 — c. An array's speed comes from being one contiguous block, and the memory after it is not guaranteed to be free. So a larger block is made elsewhere, and everything is copied. (b) is backwards: a contiguous block is exactly what keeps indexing $O(1)$.

W4-M28 — d. The array is full, so `insert_at` first resizes: 8 copies. Then all 8 elements shift one place right to free slot 0: 8 more. Then `"x"` is written. (a) forgets one of the two, and (b) counts the new element as a moved one.

W4-M29 — a. `pop(2)` returns `a[2]`, which is 30, and 40 and 50 shift left to close the gap. (c) counts from 1; (d) also counts the removed element.

W4-M30 — b. The course `DynamicArray.pop` shifts and clears a slot, but never resizes: the capacity stays 16. Shrinking at one quarter (c) is the Homework 4 extension, `ShrinkingArray` (W4-C3), not the course class.

W4-M31 — c. Resizes copy 1, 2, 4, ...; the last one happens when the size is $2^{19} = 524288$, because 2^{20} is over a million. (a) and (b) would need the array to be full at that size, which it never is; (d) is the number of resizes, not the size of the largest.

W4-M32 — d. The copies are $1 + 3 + 9 + \cdots + 3^k = (3^{k+1} - 1)/2$ with $3^k < n$, which is less than $1.5 \cdot 3^k < 1.5n$ — the general bound $n \cdot r/(r-1)$ with r = 3. (b) is true but weaker, and (c) is not even linear.

W4-M33 — a. A bound on the **total** of any sequence, shared over its operations, is amortised. (b), (c) and (d) each bound one operation: worst-case statements.

W4-M34 — b. `make_block(1 * 1.5)` asks for 1.5 slots, and the `Array` raises `TypeError: 'float' object cannot be interpreted as an integer`. A factor of 1.5 is fine in theory — (a) and (c) are wrong — it only needs rounding to a whole number, as Lecture 04 shows.

W4-M35 — c. The first append fits. The second finds the array full and calls `_resize(1 * 1)`, which copies into a new block of the **same** size, so the write to slot 1 raises `IndexError`. A factor of 1 is not growth at all, and certainly not growth by one slot (a).

W4-M36 — d. Constant-step copying is quadratic, about $n^2/(2 \times 10)$: double n and the copies multiply by about 4 (3,196,800 in fact). (b) is what doubling gives — linear.

W4-M37 — c. `__setitem__` accepts the same indices as `__getitem__`, 0 to size − 1. Slot 5 is not yet an element, so it is an `IndexError`, as it is for a Python list; writing never changes the size (a). To add an element, use `append` (d).

W4-M38 — a. After the resize to capacity 8 at size 4, the appends of elements 5, 6, 7 and 8 each paid 1 coin for their own write and saved 2: 8 coins, exactly enough to copy 8 elements. Earlier

appends' coins (b) were spent on earlier resizes; nothing is borrowed (d) — the bank never goes negative.

W4-M39 — d. Append 2 sees size 1 = capacity 1 and resizes to a 2-slot block, but `_capacity` stays 1. From then on `size == _capacity` is never true again, so there are no more resizes: append 3 writes to slot 2 of a 2-slot block and raises `IndexError`. (b) is the bug of W4-B1.

W4-M40 — b. append and `insert_at` add an element, so either may find the array full. pop never resizes in the course class (c), and `__setitem__` only overwrites an existing slot (d).

Part B — Short answer and essay

W4-E1 *(5)*

- **Definition:** the amortised cost of an operation is the maximum total cost of any sequence of n operations, divided by n. It is a guarantee over sequences, not an average over inputs.
- **Writes:** each of the n appends writes one element: n.
- **Copies:** resizes happen when the size reaches 1, 2, 4, ..., 2^k with $2^k < n$, each copying that many elements.
- **Sum:** $1 + 2 + \cdots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k < 2n$.
- **Conclusion:** total < 3n = O(n), so the amortised cost of one append is < 3 = O(1). (The accounting argument — 3 coins per append — is equally acceptable.)

W4-E2 *(4)*

- Each resize copies **everything** stored so far.
- **Constant step c:** resizes come every c appends, at sizes about c, 2c, 3c, ...; the amounts copied grow **linearly**, and a sum of n/c linearly growing terms is about $n^2/(2c)$ — $\Theta(n^2)$. A larger c only shrinks the constant.
- **Factor r:** resizes come at sizes growing **geometrically**; each one copies r times more than the last, so the whole sum is dominated by its last term, which is less than n: total $\Theta(n)$.
- The difference is linear against geometric spacing of the resizes — not the size of the step.

W4-E3 *(3)*

- **Worst case O(n):** a single append that finds the array full must copy all n elements before writing.
- **Amortised O(1):** such appends are exponentially rare; any n appends cost under 3n in total. Both are true because they measure different things — one operation, against a whole sequence.
- **When the worst case matters:** when one slow operation is itself a failure — real-time systems, a game frame, a latency-critical request. Those reserve capacity in advance.

W4-E4 *(4)*

- **Thrashing:** operations alternating around a single boundary force a resize almost every time, so each costs $O(n)$.

- **Example** (grow at full, shrink at half full): capacity 8, size 8. `append` → grow to 16 (copy 8); `pop` → size 8 = half of 16 → shrink to 8 (copy 8); `append` → grow (copy 8); and so on.
- **Rule:** grow when full, but shrink (halve) only when the size falls to **one quarter** of the capacity.
- **Why it works:** after any resize the array is about half full, so at least about n/4 cheap operations must happen before the next resize, which pays for it: amortised O(1) for any mix of appends and pops.

W4-E5 *(3)*

- A **larger factor** resizes less often (fewer total copies, e.g. under 2n for ×2) but can leave more capacity unused (up to about half).
- A **smaller factor** wastes less memory but copies more (about $n\,r/(r-1)$ in total — around 9n for r = 1.125).
- Any factor > 1 keeps append amortised O(1). CPython chooses a small one because Python programs create huge numbers of lists, most of them small, so the saved memory matters more than the extra copying.

W4-E6 *(4)*

- **Bounds:** `0 <= 1 <= 4` holds. **Full:** size 4 = capacity 4, so resize to 8: allocate an `Array(8)` and copy `a b c d` — **4 copies**.
- **Shift right**, from the end down to the gap: `d` to slot 4, `c` to 3, `b` to 2 — **3 shifts**. Then write `"x"` into slot 1; size becomes 5.
- **Result:** `a x b c d`, size 5, capacity 8; 7 elements moved, plus 1 write.
- **Why first:** the first shift writes into slot `size`, which does not exist in a full block, so shifting before resizing raises `IndexError`.

W4-E7 *(4)*

- **Stack:** the top is the **end** of the `DynamicArray`, so push is `append` (amortised $O(1)$), and pop and peek touch only the last slot ($O(1)$).
- **SlowQueue:** enqueue is `append` (amortised $O(1)$), but dequeue is `pop(0)`, which shifts every remaining element: $O(n)$ — the cost Week 7 measures.
- The structure's costs are the `DynamicArray`'s costs at the end it uses: the stack chose the cheap end; the queue cannot avoid the expensive one. (`MinHeap` adds a leaf with `append`; `Graph` keeps each neighbour list in one.)
- **Bugs travel:** these classes call your `append`, `pop` and `__getitem__`, so a wrong bound or a broken resize fails *their* tests too — the storage rule makes week 4 the foundation of weeks 6, 7, 12 and 14.

W4-E8 *(3)*

- Charge every append **3 coins**: 1 pays for its own write, 2 are saved in the bank.
- At a resize from capacity c to 2c, the c/2 appends since the previous resize have saved $2 \cdot c/2 = c$ coins — exactly enough to copy the c elements.
- So the balance never goes negative: every resize is paid for by coins already saved. Total cost ≤ 3n, amortised $O(1)$ per append.

W4-E9 *(4)*

- **The risk:** amortised $O(1)$ bounds the **total**, not each append. The append that finds the array full copies everything — with a million records, one frame does a million copies and the game stutters.
- **Fix 1 — reserve:** allocate enough capacity in advance for the expected number of records, so no resize happens during play.
- **Fix 2 — spread the copy:** when the array is half full, allocate the bigger block and copy a few old elements on each later append, so the copy is finished before the old block fills; every append is then worst-case $O(1)$.
- Also acceptable: store the records in a chain of fixed-size blocks (as `collections.deque` does) or in a linked list — worst-case $O(1)$ append, at a cost in memory and locality.

Part C — Trace the growth

W4-T1. Resizes on appends 2, 3, 5 and 9 (sizes 1, 2, 4, 8 were full). Capacity **16**, **4** resizes, **15** elements copied (1 + 2 + 4 + 8).

W4-T2. Resizes on appends **2, 3, 5, 9, 17**. Copies $1 + 2 + 4 + 8 + 16 = 31$; writes 20; total **51**; average \$51 / 20 = \$ **2.55** per append — under 3, as the proof promises.

W4-T3. Capacities 1 → 4 → 7 → 10 → 13. Resizes on appends **2, 5, 8, 11**. Final capacity **13**; copies \$1 + 4 + 7 + 10 = \$ **22**. The copies grow linearly — the signature of a constant step.

W4-T4.

Operation	Size	Capacity	Why
append(1)	1	1	room
append(2)	2	2	full → double
append(3)	3	4	full → double
append(4)	4	4	room
append(5)	5	8	full → double
pop() → 5	4	8	4 > 8 // 4 = 2
pop() → 4	3	8	3 > 2
pop() → 3	2	4	2 ≤ 2 → halve
pop() → 2	1	2	1 ≤ 4 // 4 = 1 → halve

W4-T5. Appends **2, 3, 5, 9, 17, 33**: each is one more than a power of two, because the array is full exactly when its size is a power of two. The expensive appends double their spacing each time — which is why their cost averages out.

Part D — Proofs

W4-P1. Resizes happen when the size reaches $1, 4, 16, \ldots, 4^k$ with $4^k < n$, and each copies that many elements. The copies total

$$1 + 4 + \cdots + 4^k = \frac{4^{k+1} - 1}{3} < \frac{4}{3} \cdot 4^k < \frac{4n}{3}.$$

Adding the n writes, the total is below $n + \frac{4n}{3} = \frac{7n}{3} = O(n)$, so append is amortised $O(1)$. ■ (The same proof works for any factor $r > 1$: copies $< n \cdot \frac{r}{r-1}$.)

W4-P2. Capacities are $1, 1 + c, 1 + 2c, \ldots$. A resize happens when the size equals the capacity, $1 + jc$, for every j with $1 + jc < n$; call the number of resizes m. Then $m \geq (n-1)/c$, and resize j copies $1 + jc$ elements, so

$$\text{copies} = \sum_{j=0}^{m-1} (1 + jc) = m + c\,\frac{m(m-1)}{2} \;\geq\; \frac{n-1}{c} + \frac{(n-1)(n-1-c)}{2c} = \frac{(n-1)(n+1-c)}{2c}.$$

Expanding, $\frac{(n-1)(n+1-c)}{2c} = \frac{n^2}{2c} - \frac{n}{2} + \frac{c-1}{2c} \geq \frac{n^2}{2c} - n$. For fixed c this is $\Omega(n^2)$: no constant step can make append amortised $O(1)$. ■

Part E — Find and fix the bug

W4-B1. The test should be `==`: with `>`, the array is never grown when it is exactly full, so the write goes one past the end of the `Array` — the second append raises `IndexError: Array index 1 out of range for length 1`. **Fix:** `if self._size == self._capacity:`.

W4-B2. The bound is the **capacity**, so reading an unused slot returns `None` instead of raising: with `[1, 2, 3]` stored (capacity 4), `a[3]` gives `None`. `test_index_out_of_range_raises` fails. **Fix:** `0 <= index < self._size`.

W4-B3. It grows by a **constant** (`growth` is 2, so `+ 2`), not by a factor: 1,000 appends resize about 500 times, and append is amortised $O(n)$. **Fix:** `self._resize(self._capacity * self.growth)`.

W4-B4. After shifting, the slot at the old last position still refers to the last element (and, for a pop from the end, to the removed value). The array keeps that object alive, so the garbage collector cannot reclaim it — a memory leak for large objects. **Fix:** after `self._size -= 1`, add `self._block[self._size] = None`.

Part F — Write the code

W4-C1

```
def capacity_after(n, factor=2):
    capacity, size = 1, 0
    for _ in range(n):
        if size == capacity:
            capacity *= factor
        size += 1
    return capacity
```

W4-C2

```
def copies_for(n, grow):
    capacity, size, copies = 1, 0, 0
    for _ in range(n):
        if size == capacity:
           copies += size              # the resize copies everything stored
            capacity = grow(capacity)
        size += 1
    return copies
```

`copies_for(100_000, lambda c: 2 * c)` is 131,071 and `copies_for(100_000, lambda c: c + 1)` is 4,999,950,000 — Lecture 04's numbers.

W4-C3

```
class ShrinkingArray:
    def __init__(self):
        self._block = Array(1)
        self._size = 0
        self.resize_count = 0

    @property
    def capacity(self):
        return len(self._block)

    def _resize(self, capacity):
        block = Array(capacity)
        for i in range(self._size):
            block[i] = self._block[i]
        self._block = block
        self.resize_count += 1

    def append(self, value):
        if self._size == self.capacity:
            self._resize(2 * self.capacity)
```

```
        self._block[self._size] = value
        self._size += 1

    def pop(self):
        if self._size == 0:
            raise IndexError("pop from empty array")
        self._size -= 1
        value = self._block[self._size]
       self._block[self._size] = None          # let the object be collected
        if self._size <= self.capacity // 4 and self.capacity > 1:
            self._resize(self.capacity // 2)
        return value

    def __len__(self):
        return self._size

    def __getitem__(self, index):
        if not 0 <= index < self._size:
            raise IndexError(index)
        return self._block[index]
```

The thrashing test: it fills the array to size 8, capacity 8, then alternates append and pop. The first append grows to 16 (size 9); the pop leaves size 8, which is above 16 // 4 = 4, so nothing happens — and the same for every later pair: one resize in total. With a half rule, size 8 ≤ 16 // 2 would shrink back to 8, and the next append would grow again: a resize on every operation, and the test fails.